



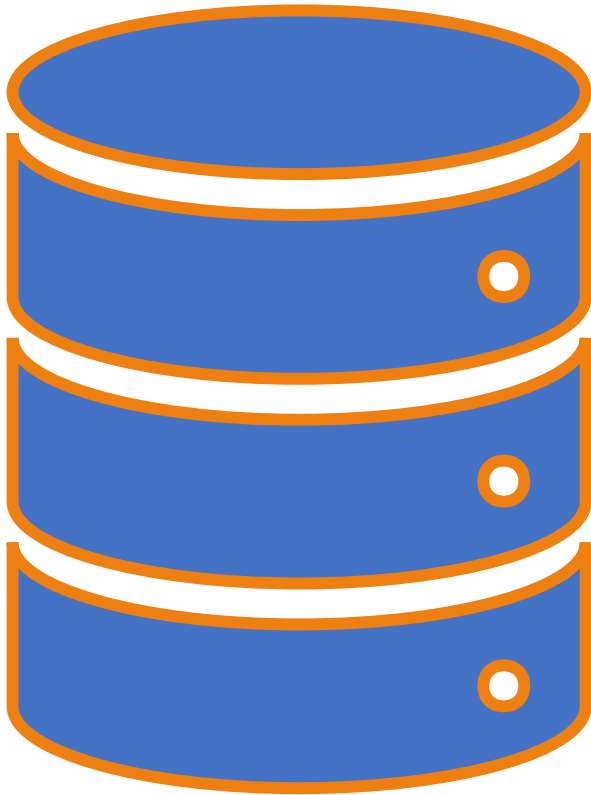
Bancos e Armazéns de Dados

*Profª Me. Claudia Maria Sodero
Salles*



Aviso legal

- Este conteúdo é parte integrante da disciplina “Bancos de Dados e Armazém de Dados”, do curso de Ciência de Dados da FATEC Baixada Santista – Rubens Lara
- Seu compartilhamento, divulgação, publicação e utilização está sujeito à autorização da professora Claudia Maria Soderro Salles, e está completamente proibido fora do âmbito da disciplina
- Santos, 2º semestre 2026



A linguagem SQL

*Executando consultas e manipulando
registros*

Linguagem SQL

Estrutura da Linguagem

Comandos Básicos

Funções

Na prática

DML - Linguagem de Manipulação de Dados - Define os comandos utilizados para manipulação de dados no banco ;

- SELECT, INSERT, UPDATE e DELETE

DDL - Linguagem de Definição de Dados - Define os comandos utilizados para criação de tabelas, views, índices, atualização dessas estruturas, assim como a remoção;

- CREATE, ALTER e DROP

DCL - Linguagem de Controle de Dados - Define os comandos utilizados para controlar o acesso aos dados do banco, adicionando e removendo permissões de acesso;

- GRANT e REVOKE

DTL - Linguagem de Transação de Dados - Define os comandos utilizados para gerenciar as transações executadas no banco de dados, como iniciar uma transação, confirmá-la ou desfazê-la.

- BEGIN, COMMIT e ROLLBACK

Estrutura da Linguagem

DDL – Data Definition Language



Comandos Básicos

- Criação do Banco de Dados

CREATE DATABASE

<nome_do_banco>;

- Usar um Banco de Dados Específico

USE <nome_do_banco>;

- Criação de Tabelas

CREATE TABLE <nome_da_tabela> (

<nome_do_atributo1> <tipo_de_dado> [NULL/NOT NULL],

<nome_do_atributo2> <tipo_de_dado> [NULL/NOT NULL],

...

<nome_atributoN> <tipo_de_dado> [NULL/NOT NULL]);

- Exclusão de Tabelas

DROP TABLE <nome_da_tabela>;

DML – Data Manipulation Language



Comandos Básicos

- Inserção de registros nas tabelas

```
INSERT INTO <nome_tabela>  
(<nome_atributo1>, ..., <nome_atributo>) VALUES  
(valor1, ..., valorN);
```

- Seleção de registros nas tabelas

```
SELECT <nome_atributo1>, ..., <atributoN> FROM  
<nome_tabela> [WHERE condição]
```

- Exclusão de registros nas tabelas

```
DELETE <nome_da_tabela>;
```

- Alteração de registros nas tabelas

```
UPDATE <nome_da_tabela> SET <NOME_DO_ATRIBUTO> =  
valor;
```

DESAFIO:

Qual a diferença entre **DELETE FROM <TABELA>**, **DROP <TABELA>** e **TRUNCATE TABLE <TABELA>** ????

Operadores Condicionais



Operadores Condicionais

- = (igual)
- != ou <> (diferente)
- > (maior que)
- < (menor que)
- >= (maior ou igual a)
- <= (menor ou igual a)
- LIKE (similar a)
- BETWEEN (entre X e Y)
- IN (vários valores dentro da lista)
- AND (e)
- OR (ou)
- NOT (negação)
- IS (valores iguais)

Exemplo

CAD_FUNCIONARIO					
id_func	nm_funcionario	cpf	dt_nasc	id_sexo	id_graduacao
1	João José	111.111.111-11	01/03/1997	1	3
2	Maria Francisca	222.222.222-22	05/02/1978	2	3
3	Joaquim da Silva	333.333.333-33	07/04/1990	1	3
4	Manoel Vieira	444.444.444-44	03/08/1985	1	2
5	Isabella Meira	555.555.555-55	04/07/1973	2	4
6	Janaína Costa	666.666.666-66	05/08/1958	2	2
7	Irineu Mathias	777.777.777-77	09/10/1995	1	3
8	Wellington Dias	888.888.888-88	25/08/1970	1	2
9	Soraia Costa	999.999.999-99	03/11/1988	2	4

Quais os funcionários têm mais de 40 anos? Mostre o nome e a idade.

Exem

CAD_FUNCIONARIO					
id_func	nm_funcionario	cpf	dt_nasc	id_sexo	id_graduacao
1	João José	111.111.111-11	01/03/1997	1	3
2	Maria Francisca	222.222.222-22	05/02/1978	2	3
3	Joaquim da Silva	333.333.333-33	07/04/1990	1	3
4	Manoel Vieira	444.444.444-44	03/08/1985	1	2
5	Isabella Meira	555.555.555-55	04/07/1973	2	4
6	Janaína Costa	666.666.666-66	05/08/1958	2	2
7	Irineu Mathias	777.777.777-77	09/10/1995	1	3
8	Wellington Dias	888.888.888-88	25/08/1970	1	2
9	Soraia Costa	999.999.999-99	03/11/1988	2	4

Quais os funcionários têm mais de 40 anos? Mostre o nome e a idade.

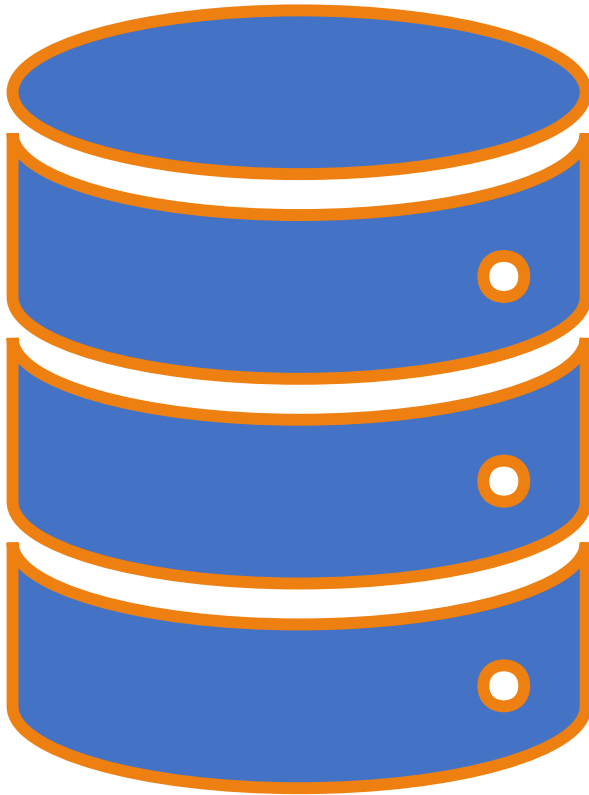
```
SELECT nm_funcionario AS nome, (year(getdate()) – year(dt_nasc)) AS idade
FROM cad_funcionario
WHERE (year(getdate()) – year(dt_nasc)) > 40
```

Dica bacana

Guia completo SQL, com muitos artigos

<https://www.devmedia.com.br/guia/guia-completo-de-sql/38314>





A linguagem SQL

*REALIZANDO CONSULTAS EM MÚLTIPLAS
TABELAS*

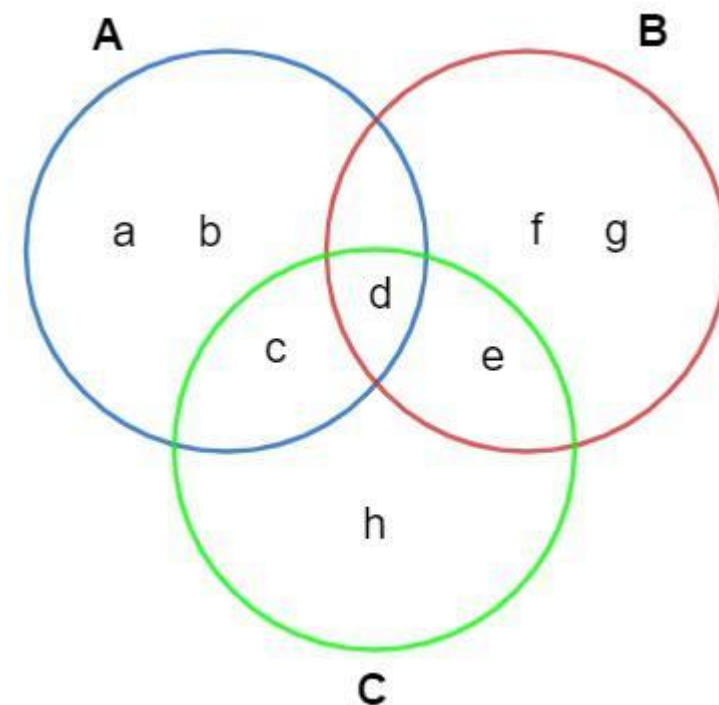
O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- A maioria dos bancos de dados tem múltiplas tabelas.
- O que é uma junção e como ela funciona - Correspondência de linhas em uma tabela com linhas correspondentes em outra tabela e misturá-las.
- Não é sempre baseado em um relacionamento entre uma chave primária e uma chave estrangeira.
- A chave primária pode ter múltiplas colunas.
- Muitas das coisas que faremos no Transact-SQL são baseadas na teoria baseada em conjuntos ou teoria dos conjuntos em matemática.
- É útil, especialmente quando você está pensando em junções, pense nas coisas em termos de diagramas de Venn. Coleções de coisas.

O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- Diagramas de Venn

O diagrama de Venn, também conhecido como diagrama de Venn-Euler, é uma maneira de representar graficamente um conjunto, para isso utiliza-se uma linha fechada que não possui auto intersecção e representa-se os elementos do conjunto no interior dessa linha.



"Veja mais sobre "Diagrama de Venn" em:
<https://brasilecola.uol.com.br/matematica/diagrama-de-venn.htm>

O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- O que é um produto cartesiano?
 - multiplicação entre pares ordenados envolvendo conjuntos distintos.

Por exemplo, temos o conjunto “A” formado pelos seguintes elementos {1, 2, 3, 4} e o conjunto “B” formado pelos elementos {2, 3}



$A = \{1, 2, 3, 4\}$
 $B = \{2, 3\}$

$A \times B: \{(1, 2); (2, 2); (3, 2); (4, 2); (1, 3); (2, 3); (3, 3); (4, 3)\}$

$B \times A: \{(2, 1); (2, 2); (2, 3); (2, 4); (3, 1); (3, 2); (3, 3); (3, 4)\}$

FUNDAMENTOS NO USO DA SINTAXE

- Usando junções, você pode recuperar dados de duas ou mais tabelas com base em relacionamentos lógicos entre as tabelas.
- Uma condição de junção define a forma como duas tabelas estão relacionadas em uma consulta por:
 - Especificando a coluna de cada tabela a ser usada para a junção.
 - Uma condição de junção típica especifica uma chave estrangeira de uma tabela e sua chave associada na outra tabela.
 - Especificar um operador lógico (por exemplo, = ou <>) a ser usado na comparação de valores das colunas.
- As condições de junção combinam-se com as condições de pesquisa WHERE e HAVING para controlar as linhas selecionadas
- Sintaxe básica

SELECT [atributo(s)] **FROM** tabela
JOIN tabela **ON** [Join condition]

FUNDAMENTOS NO USO DA SINTAXE

```
SELECT ProductID, Purchasing.Vendor.BusinessEntityID, Name
FROM Purchasing.ProductVendor JOIN Purchasing.Vendor
    ON (Purchasing.ProductVendor.BusinessEntityID = Purchasing.Vendor.BusinessEntityID)
WHERE StandardPrice > $10
    AND Name LIKE N'F%'
GO
```

O **select** retorna as informações do produto e do fornecedor para qualquer combinação de peças fornecidas por uma empresa para a qual o nome da empresa começa com a letra F e o preço do produto é superior a US\$ 10.

Quando várias tabelas são referenciadas em uma única consulta, todas as referências de coluna devem ser inequívocas. Por exemplo, as tabelas ProductVendor e Vendor têm uma coluna chamada BusinessEntityID. Qualquer nome de coluna que está duplicado entre duas ou mais tabelas referenciadas na consulta deve ser qualificado com o nome da tabela.

O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- PRINCIPAIS TIPOS DE JOIN (junção)
 - INNER JOIN
 - LEFT JOIN
 - RIGHT JOIN
 - CROSS JOIN

O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- INNER JOIN

Uma junção interna é uma junção na qual os valores nas colunas que estão sendo unidas são comparados usando um operador de comparação.

```
SELECT *  
FROM Employee AS e  
INNER JOIN Person AS p  
ON e.BusinessEntityID = p.BusinessEntityID  
ORDER BY p.LastName
```

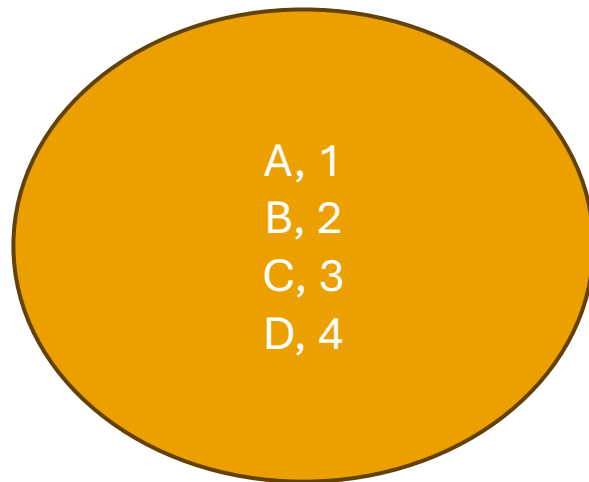


Retorna todas as colunas em ambas as tabelas e retorna apenas as linhas para as quais há um valor igual na coluna de junção.

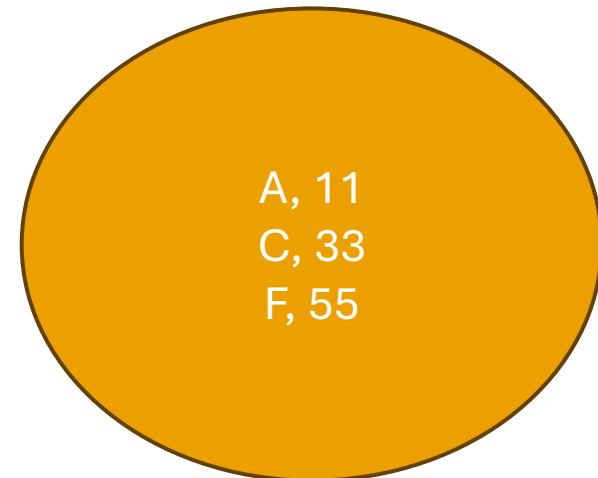
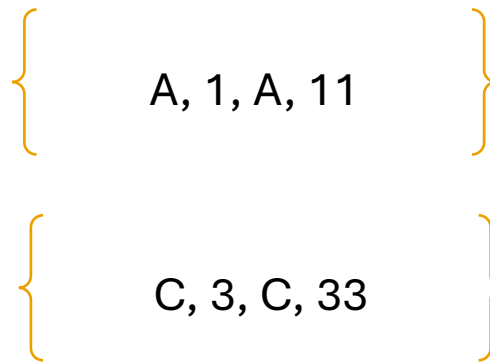
O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- INNER JOIN

```
SELECT *  
FROM Employee AS e  
INNER JOIN Person AS p  
ON e.BusinessEntityID = p.BusinessEntityID  
ORDER BY p.LastName
```



e



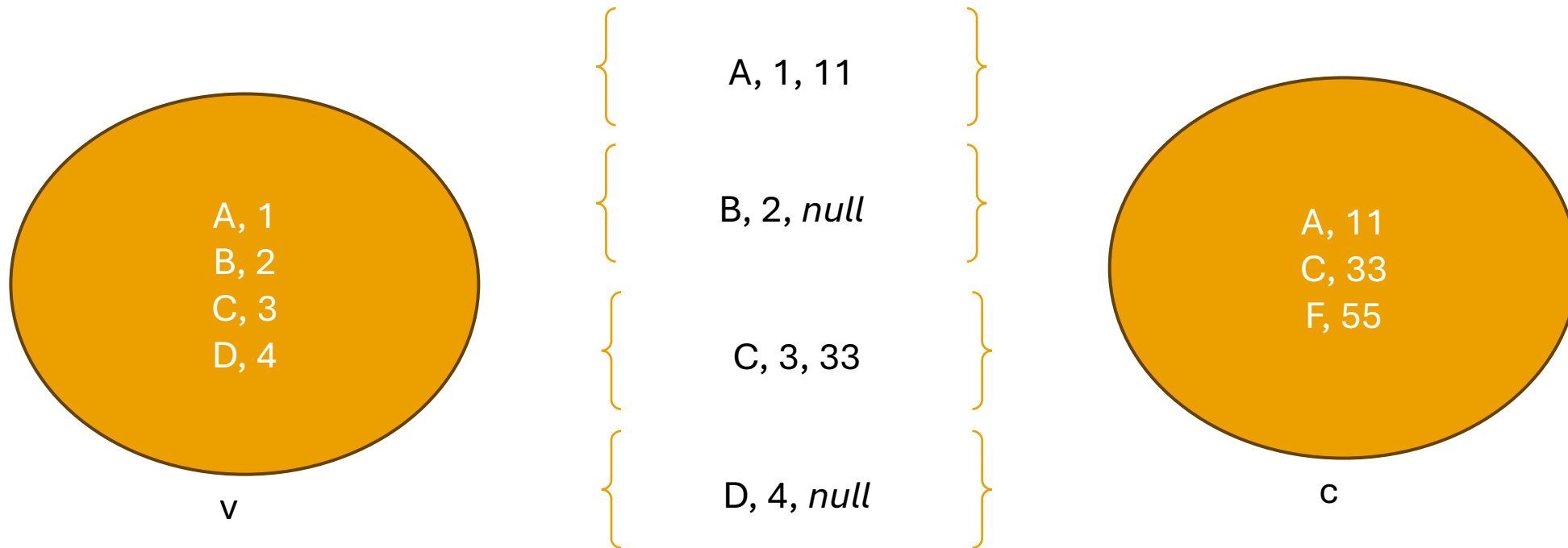
p

-
- As junções internas retornam linhas somente quando há pelo menos uma linha de ambas as tabelas que corresponde à condição de junção.
 - A devolutiva é um subconjunto de linhas com estas igualdades
 - Mas quando queremos todas as vendas de uma loja e, quando identificado o cliente, trazer os dados do cliente? Pense que nem todas as vendas há a identificação do cliente! Se usarmos INNER JOIN, só teremos as vendas com o cliente identificado!

O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- LEFT JOIN

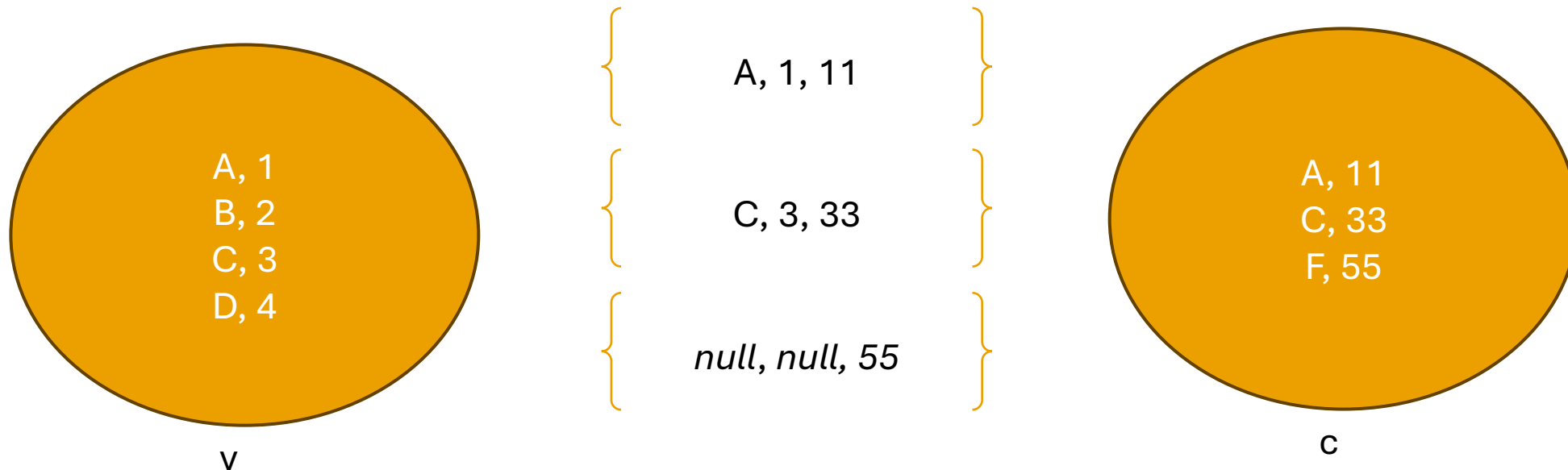
```
SELECT v.*, c.nome  
FROM venda v  
LEFT JOIN cliente c  
ON v.ClientID = c.ClientID
```



O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- RIGHT JOIN

```
SELECT v.*, c.nome  
FROM venda v  
RIGHT JOIN cliente c  
ON v.ClientID = c.ClientID
```



O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

- CROSS JOIN

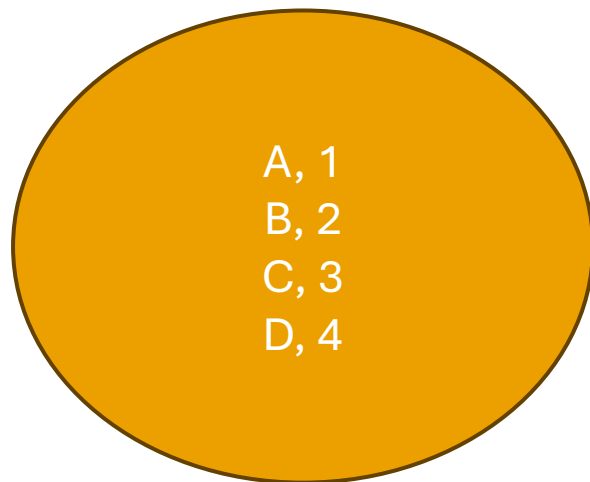
Uma junção cruzada produz o produto cartesiano das tabelas envolvidas na junção.

O tamanho de um conjunto de resultados de produtos cartesianos é o número de linhas na primeira tabela multiplicado pelo número de linhas na segunda tabela.

```
SELECT v.*, c.*  
FROM vendas v  
CROSS JOIN cliente t;
```

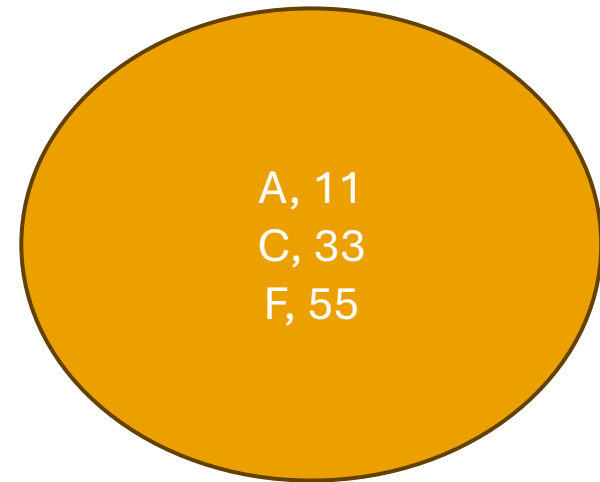
O QUE É UM “JOIN” E QUANDO DEVE SER USADO?

A, 1, A, 11	B, 2, A, 11	C, 3, A, 11	D, 4, A, 11
A, 1, C, 33	B, 2, C, 33	C, 3, C, 33	D, 4, C, 33
A, 1, F, 55	B, 2, F, 55	C, 3, F, 55	D, 4, F, 55



V

```
SELECT v.*, c.*  
FROM vendas v  
CROSS JOIN cliente t;
```



C

RESUMINDO....

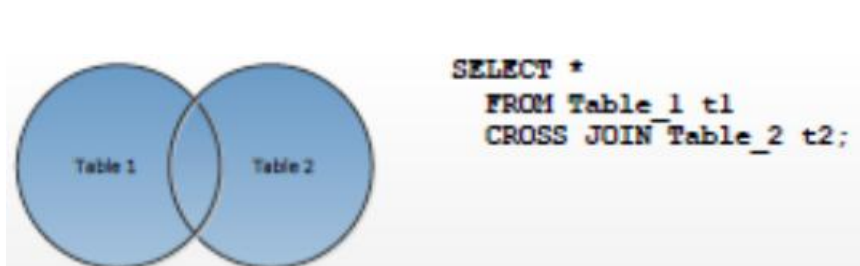
INNER JOIN



LEFT JOIN



CROSS JOIN



RIGHT JOIN

